

# A Risk-Adaptive Late-Binding Decryption Framework for Sensitive Data Exposure in Secure Applications

**Abstract**—Sensitive database fields may be decrypted earlier than necessary during backend processing, increasing plaintext exposure even when a request is legitimately authorized. Existing encryption and access-control mechanisms primarily protect stored data or determine authorized access, but they do not necessarily provide a separate post-authorization control over when and how much protected information is materialized as plaintext. This paper proposes a Risk-Adaptive Late-Binding (RALB) decryption framework that keeps protected fields encrypted after object-relational mapping (ORM) retrieval and performs decryption only after mandatory security checks and contextual risk evaluation. Transparent Data Encryption (TDE) protects database storage at rest, while application-level encryption (ALE) preserves sensitive fields as ciphertext until the RALB policy decision. A context builder constructs trusted server-derived features, an ML-based predictor estimates LOW, MEDIUM, or HIGH exposure risk, and a policy engine maps the predicted class to full authorized-field decryption, partial decryption with masking, or denial, respectively. The framework was evaluated using a synthetic dataset of 10,000 access events with six raw contextual features labeled by a fixed reference policy. Random Forest and JRip both achieved 98.00% accuracy and a weighted F1-score of 0.980, with Macro-F1 scores of 0.947 and 0.945, respectively. Both achieved a HIGH-class recall of 0.935, giving a conservative CUR upper bound of 0.065. These results demonstrate accurate approximation of the predefined exposure-risk policy, while runtime plaintext-exposure reduction remains an implementation-level verification target.

**Index Terms**—application security, application-level encryption (ALE), data privacy, late-binding decryption, plaintext exposure, risk-adaptive access control, transparent data encryption (TDE), object-relational mapping (ORM)

## I. INTRODUCTION

Data privacy is a fundamental requirement in modern digital systems because applications store and process sensitive personal, financial, medical, and organizational information [1]. Protecting such information requires not only controlling access but also limiting unnecessary exposure during application processing. Encryption protects sensitive data while they remain stored or transmitted [2], [3], and authenticated encryption mechanisms such as AES-GCM provide confidentiality and integrity [4]. However, encrypted data must eventually become plaintext for legitimate processing, and early decryption can increase the duration of sensitive-data exposure [3]. Therefore, data protection should consider not only whether information is encrypted, but also when and how much sensitive information becomes plaintext during application execution.

ORM-based applications introduce a particular plaintext-exposure risk because protected database records may be loaded

before all security and processing decisions are completed [5], [6]. Access-control mechanisms such as Role-Based Access Control (RBAC), Attribute-Based Access Control (ABAC), Zero Trust, and risk-adaptive access control determine whether a request is allowed based on identity, attributes, policies, and contextual conditions [7], [8], [9], [10], [11], [12], [13], [14]. However, successful authorization does not directly determine how much protected information should become plaintext after access is granted.

A security gap therefore exists between granting access to a resource and controlling how much of its protected data becomes plaintext. A legitimately authorized request may still cause more sensitive fields to be decrypted than are required for the requested operation [6], [3]. Therefore, reducing unnecessary plaintext materialization can provide an additional application-layer security boundary without changing the permissions established by the underlying access-control mechanism.

This paper proposes a *Risk-Adaptive Late-Binding (RALB) decryption framework*. The framework keeps protected fields encrypted after ORM retrieval and delays plaintext materialization until mandatory security checks and contextual risk evaluation have been completed. Eligible requests are classified as LOW, MEDIUM, or HIGH risk using contextual information, and a policy engine maps these classes to full authorized-field decryption, limited masked exposure, or no protected-field decryption, respectively. Thus, the risk layer can reduce plaintext exposure without increasing the permissions already granted by the authorization system.

The main contributions of this work are as follows:

- A late-binding decryption approach that separates ORM object retrieval from field-level plaintext materialization.
- A risk-adaptive exposure policy that supports full, partial, or no protected-field decryption according to the predicted risk level.
- An application-layer framework combining mandatory security checks, contextual risk prediction, a policy engine, and AES-256-GCM field-level protection.
- A controlled proof-of-concept evaluation using 10,000 synthetic access events and multiple machine-learning classifiers.

## II. RELATED WORK

Encryption is widely used to protect sensitive information in database and application systems. NIST specifies AES-

GCM as an authenticated-encryption mode that provides confidentiality and authentication [4]. Shmueli *et al.* examined major database-encryption design issues, including encryption location, granularity, key management, and integration overhead [15]. CryptDB demonstrated another direction by supporting database queries while keeping much of the stored information encrypted [16]. These approaches strengthen data protection, but their primary focus is storage confidentiality or encrypted-data processing rather than application-level decisions about when individual protected fields should become plaintext.

Access-control research addresses the authorization side of the problem. Ferraiolo and Kuhn introduced RBAC for controlling permissions through organizational roles [7], and Sandhu *et al.* later formalized a family of RBAC models [8]. ABAC extends authorization by evaluating subject, object, action, and environmental attributes [9]. Zero Trust further removes implicit trust and emphasizes repeated authentication and authorization around protected resources [10]. Risk-adaptive models incorporate additional contextual or situational information. Kandala *et al.* combined attributes with risk-adaptive access control [11]. Shaikh *et al.* proposed dynamic risk-based decision methods [12], while Khambhammettu *et al.* developed a framework for estimating risk associated with access requests [13]. Dos Santos *et al.* extended risk-based authorization to cloud environments [17]. A systematic review by Atlam *et al.* further showed that risk-based models mainly use contextual and real-time information to make dynamic access decisions [14]. Recent work also reports growing use of contextual security and machine learning for adaptive access control and identity validation [18], [19].

Across these studies, the main control point remains the access decision itself [11], [12], [13], [17], [14]. Existing work also demonstrates field-level encryption, encrypted processing, masking, and fine-grained access-control mechanisms; therefore, this paper does not claim field-level protection itself as new. The narrower gap considered here is post-authorization plaintext materialization in ORM-based backend processing: how much of an already authorized protected record should become plaintext, and when. The proposed framework addresses this gap through risk-adaptive late-binding decryption, in which protected fields remain ciphertext after ORM retrieval and the permitted exposure level is selected only after mandatory authorization and contextual risk evaluation.

### III. RISK-ADAPTIVE LATE-BINDING (RALB) DECRYPTION FRAMEWORK

The proposed RALB framework delays protected-field decryption until mandatory security checks and contextual risk evaluation are completed. As shown in Fig. 1, request  $q$  passes through the Security Gate and Business Logic/services before querying a TDE-enabled database. Transparent Data Encryption (TDE) protects database storage at rest, while sensitive fields remain protected by application-level encryption (ALE) and are returned as ciphertext  $C_j^{(1)}$ . The Context Builder produces  $\mathbf{x}_q$ , the ML-Based Risk Prediction block estimates

LOW, MEDIUM, or HIGH risk, and the Policy Engine controls full decryption, partial decryption with masking, or denial. The selected output follows the single response path to the Response Builder. The ML model receives trusted context  $\mathbf{x}_q$ , not ciphertext or plaintext.

#### A. Security Gate and Ciphertext-Preserving Data Retrieval

For a request  $q$ , let  $A_n(q)$ ,  $A_z(q)$ ,  $A_t(q)$ ,  $A_s(q)$ , and  $A_o(q) \in \{0, 1\}$  denote authentication, authorization, tenant validation, scope validation, and any applicable ownership rule, respectively. The mandatory security gate is

$$G(q) = A_n(q) \wedge A_z(q) \wedge A_t(q) \wedge A_s(q) \wedge A_o(q). \quad (1)$$

Let  $E_{\text{ctx}}(q)$  indicate a context, schema, prediction, or policy processing failure, and let  $N_{\text{dec}}(q)$  be the number of protected fields decrypted. The framework follows a fail-closed rule:

$$[G(q) = 0] \vee [E_{\text{ctx}}(q) = 1] \Rightarrow \text{DENY}(q) \wedge N_{\text{dec}}(q) = 0. \quad (2)$$

Only after  $G(q) = 1$  may the business/service layer query the required record. TDE protects the underlying database files and backups, but it is transparent to application query processing and does not control application-level plaintext exposure [15]. Sensitive fields therefore remain under ALE, so the ORM materializes  $C_j^{(1)}$  and only the minimum non-sensitive metadata needed for processing. Protected values are not converted to plaintext at retrieval time; retrieval is separated from the later RALB decryption decision.

#### B. Context Builder and Feature Representation

The Context Builder in Fig. 1 derives six raw server-observable features for each eligible request: user role, resource sensitivity, office-hour status, record-owner match, recent request count, and failed-authentication count. The three-valued role feature is one-hot encoded, producing the 8-dimensional model input

$$\mathbf{x}_q = [i_1, i_2, i_3, r, t, a, v, f]^T \in \mathbb{R}^8, \quad (3)$$

where  $i_1$ ,  $i_2$ , and  $i_3$  encode Customer, Moderator, and Admin, respectively;  $r$  is resource sensitivity;  $t$  is office-hour status;  $a$  is record-owner match;  $v$  is recent request count; and  $f$  is the failed-authentication count. All values are obtained from trusted server-side state rather than client-supplied claims. The same encoding function is used for training, validation, testing, and runtime inference:

$$\phi_{\text{train}}(q) = \phi_{\text{eval}}(q) = \phi_{\text{run}}(q). \quad (4)$$

Invalid categories, missing mandatory context, or out-of-schema values invoke the fail-closed behavior in Eq. (2).

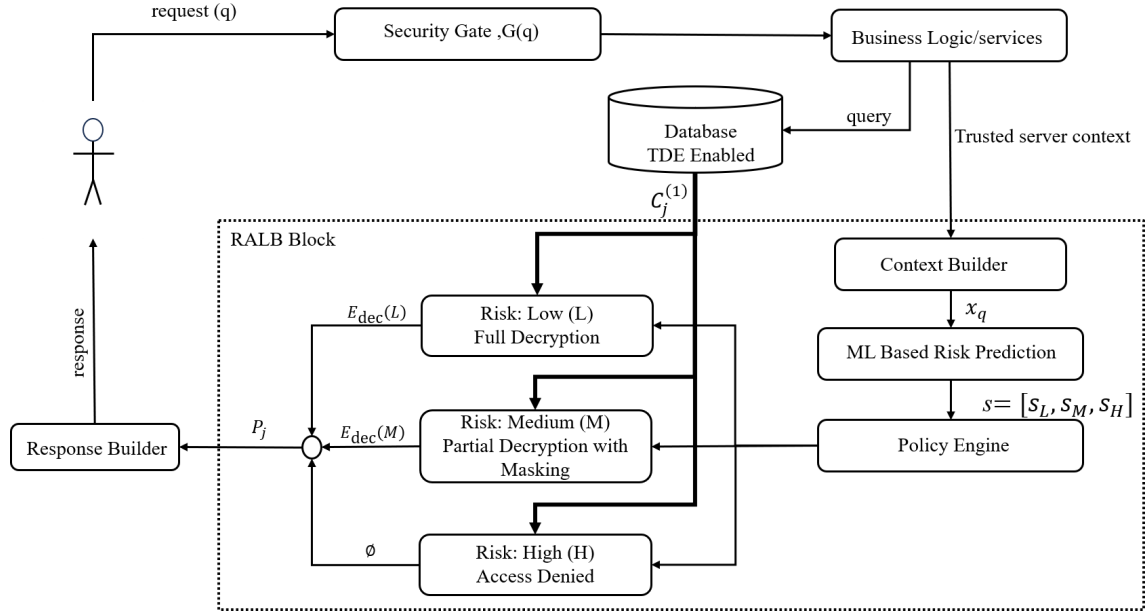


Fig. 1. One-way RALB data flow: TDE protects database storage, ALE preserves sensitive fields as ciphertext, and the policy controls decryption.

TABLE I  
SUMMARY OF THE GENERATED DATASET

| Property             | Description                   |
|----------------------|-------------------------------|
| Samples / seed       | 10,000 / 42                   |
| Raw features         | 6 contextual features         |
| Encoded dimension    | 8 after role one-hot encoding |
| Risk classes         | LOW, MEDIUM, HIGH             |
| User roles           | Customer, Moderator, Admin    |
| Resource sensitivity | LOW, MEDIUM, HIGH             |
| Request-count window | Previous 60 s                 |
| Failed-login window  | Previous 24 h                 |

TABLE II  
TRAINING-SET CLASS DISTRIBUTION

| Risk Class   | Training Samples | Percentage  |
|--------------|------------------|-------------|
| LOW          | 5,945            | 84.93%      |
| MEDIUM       | 406              | 5.80%       |
| HIGH         | 649              | 9.27%       |
| <b>Total</b> | <b>7,000</b>     | <b>100%</b> |

TABLE III  
FIXED REFERENCE EXPOSURE-RISK POLICY

| Tier   | Predefined conditions (any condition)                                                                                                                                    |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HIGH   | $I_1 \wedge \neg A \wedge R \in \{M, H\}; I_1 \wedge V > 15; I_2 \wedge R = H; I_2 \wedge V > 25 \wedge \neg A; I_3 \wedge \neg T \wedge R = H \wedge V > 20; F \geq 5.$ |
| MEDIUM | $I_1 \wedge \neg A \wedge R = L; I_1 \wedge 9 \leq V \leq 15; I_2 \wedge \neg T \wedge R = M; I_3 \wedge F \in \{3, 4\}; (I_2 \vee I_3) \wedge V > 18.$                  |
| LOW    | All remaining valid contexts.                                                                                                                                            |

### C. Dataset Construction and Training Data

As an offline support stage, a synthetic access-event dataset of 10,000 samples was generated with a fixed random seed of 42 because no public dataset directly represents application-layer decryption decisions together with the runtime security context required by this study. Each sample corresponds to one access event and contains the six raw contextual features defined above. The dataset contains normal and suspicious patterns, including cross-record access, high request rates, privileged off-hours access, and repeated authentication failures.

Approximately 75% of the events represent customers, 20% moderators, and 5% administrators. Resource sensitivity is distributed approximately as 60% LOW, 30% MEDIUM, and 10% HIGH. Recent request counts mainly follow a Poisson distribution with  $\lambda = 4$ , with additional high-frequency events included to exercise abnormal bulk-access conditions in the reference policy. The synthetic generator is intended to provide a controlled policy-approximation test rather than to estimate the prevalence of behaviors in a production system.

### D. Reference Risk Policy and Label Generation

For offline label generation, a deterministic reference policy  $\mathcal{P}(\cdot)$  is used to assign the ground-truth risk label for each generated event:

$$c_q = \mathcal{P}(\mathbf{x}_q), \quad c_q \in \{L, M, H\}, \quad (5)$$

where  $L$ ,  $M$ , and  $H$  denote LOW, MEDIUM, and HIGH risk. Let  $I_1$ ,  $I_2$ , and  $I_3$  denote Customer, Moderator, and Admin;  $R$  the resource sensitivity;  $T$  the office-hour condition;  $A$  the owner-match condition;  $V$  the recent request count; and  $F$  the failed-authentication count. HIGH-risk conditions are evaluated before MEDIUM-risk conditions, and all remaining valid contexts are labeled LOW.

Here,  $\neg T$  denotes a request outside the predefined office-hour condition. The reference policy is used as a controlled

labeling oracle for the ML experiment. It is distinct from the mandatory security gate: a request that fails Eq. (1) is rejected before risk prediction and never gains access through the ML layer.

The deterministic policy is not presented as inferior to ML and could be executed directly when the policy is fixed and fully specified. In this proof-of-concept, ML is evaluated as a policy-approximation layer to test whether contextual decisions can be learned with low security-critical error. This establishes a controlled baseline for future settings in which exposure risk may be learned from audited operational decisions or from policies that evolve beyond a compact rule set. Accordingly, the reported classification metrics measure fidelity to the reference policy, not superiority over direct rule execution.

#### E. ML-Based Risk Prediction

The ML-Based Risk Prediction block in Fig. 1 receives  $\mathbf{x}_q$  from the Context Builder and forwards the score vector  $\mathbf{s} = [s_L, s_M, s_H]$  to the Policy Engine. Four candidate classifiers available in Weka [20] are considered: OneR [21], Random Forest [22], AdaBoost [23], and JRip (RIPPER) [24]:

$$\mathcal{M} = \{\text{OneR}, \text{RF}, \text{AdaBoost}, \text{JRip}\}. \quad (6)$$

For classifier  $M \in \mathcal{M}$ , the model produces class scores  $\mathbf{s}_q^{(M)} = [s_L, s_M, s_H]$  and the predicted class is

$$\hat{c}_q^{(M)} = \arg \max_{k \in \{L, M, H\}} s_k, \quad \hat{c}_q^{(M)} \in \{L, M, H\}. \quad (7)$$

The predicted labels are compared with the reference labels in Eq. (5). Validation data are used to compare candidate predictors, while the held-out test set is used only for final performance reporting. This proof-of-concept does not fix a single deployment classifier; Random Forest and JRip are retained as the strongest candidates in the reported evaluation. If model inference fails or produces an invalid output, the request follows the fail-closed rule in Eq. (2).

A security-sensitive error measure is the HIGH-to-LOW critical underclassification rate (CUR):

$$\text{CUR}(M) = \frac{N_{H \rightarrow L}(M)}{N_H}, \quad (8)$$

where  $N_{H \rightarrow L}(M)$  is the number of HIGH-risk samples predicted as LOW and  $N_H$  is the number of reference HIGH-risk samples. When the complete HIGH-row confusion counts are unavailable, HIGH recall  $R_H$  provides only the conservative upper bound

$$\text{CUR}(M) \leq 1 - R_H(M) = \frac{N_{H \rightarrow M}(M) + N_{H \rightarrow L}(M)}{N_H}. \quad (9)$$

Therefore,  $1 - R_H$  is reported as a CUR upper bound rather than as an observed HIGH-to-LOW rate unless the exact  $H \rightarrow L$  count is available.

#### F. Risk-Adaptive Decryption Policy and Response Construction

The Policy Engine in Fig. 1 converts the predicted risk class into the permitted exposure action. TDE remains the storage-at-rest layer, whereas ALE uses AES-256-GCM field-level authenticated encryption for sensitive fields. RALB controls this ALE decryption stage and retains protected values as  $C_j^{(1)}$  until the policy decision. For plaintext field  $P_j$ , the stored application-level ciphertext is

$$C_j^{(1)} = \text{Enc}_{K, N_j}^{\text{GCM}}(P_j), \quad (10)$$

where  $K$  is the field-encryption key and  $N_j$  is a nonce that is never reused with the same key. Only  $C_j^{(1)}$  is stored and retrieved by the ORM. For the selected runtime predictor, let  $\hat{c}_q$  denote the predicted risk class. The policy mapping is

$$\pi(\hat{c}_q) = \begin{cases} \text{FULL DECRYPT}, & \hat{c}_q = L, \\ \text{PARTIAL DECRYPT/MASK}, & \hat{c}_q = M, \\ \text{ACCESS DENIED}, & \hat{c}_q = H. \end{cases} \quad (11)$$

The LOW, MEDIUM, and HIGH branches in Fig. 1 are mutually exclusive policy actions; only the selected branch can release protected data. For a field selected for release, plaintext materialization occurs only after this policy decision:

$$P_j = \text{Dec}_{K, N_j}^{\text{GCM}}(C_j^{(1)}). \quad (12)$$

Authentication-tag or key-access failure invokes the fail-closed rule in Eq. (2). The maximum number of protected fields that may be decrypted is bounded by

$$0 = B_{\text{dec}}(H) \leq B_{\text{dec}}(M) = 2 \leq B_{\text{dec}}(L) = 7, \quad (13)$$

$$N_{\text{dec}}(q) \leq B_{\text{dec}}(\hat{c}_q).$$

Because each released field requires one authenticated-decryption operation,

$$N_{\text{crypto}}(q) = N_{\text{dec}}(q). \quad (14)$$

Let  $A_{\text{AuthZ}}(q)$  be the field set permitted by the mandatory authorization logic and  $A_{\text{Risk}}(\hat{c}_q)$  the field set permitted by the risk tier. The final releasable field set is

$$A_q = \begin{cases} \emptyset, & G(q) = 0 \vee E_{\text{ctx}}(q) = 1, \\ A_{\text{AuthZ}}(q) \cap A_{\text{Risk}}(\hat{c}_q), & G(q) = 1. \end{cases} \quad (15)$$

Therefore,

$$A_q \subseteq A_{\text{AuthZ}}(q), \quad A_{\text{Risk}}(H) \subseteq A_{\text{Risk}}(M) \subseteq A_{\text{Risk}}(L). \quad (16)$$

Thus, risk adaptation can reduce plaintext exposure but cannot extend the permissions established by the mandatory access-control layer. The LOW and MEDIUM branches forward permitted plaintext  $P_j$  (with masking for MEDIUM) to the Response Builder, whereas the HIGH branch returns  $\emptyset$ .

### G. Evaluation and Security Verification

Classification performance is measured using accuracy, class-wise precision, recall, F1-score, Macro-F1, weighted F1, HIGH-class recall, CUR, and confusion analysis. For each class,

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}. \quad (17)$$

Security verification focuses on fail-closed enforcement, schema consistency, TDE-enabled storage, ciphertext-preserving ALE/ORM retrieval, authenticated AES-GCM decryption, and the field-exposure budget. The required invariants include

$$G(q) = 0 \Rightarrow N_{\text{dec}}(q) = 0, \quad \hat{c}_q = H \Rightarrow N_{\text{dec}}(q) = 0, \quad (18)$$

together with Eqs. (13) and (15). Audit records contain only contextual state, predicted risk level, policy action, and necessary system metadata; plaintext values, cryptographic keys, secrets, and unnecessary identifiers are excluded.

## IV. RESULTS AND DISCUSSION

A stratified 70/15/15 split provides 7,000 training, 1,500 validation, and 1,500 test events. Only the training set is used to fit the candidate classifiers; validation data support model comparison, and the test set remains held out for final reporting. Table II reports the LOW, MEDIUM, and HIGH class composition of the training set.

### A. Classification Performance

The completed test-set evaluation uses 1,500 samples: 1,274 LOW, 87 MEDIUM, and 139 HIGH cases. Table IV summarizes the overall classification and security-sensitive performance.

Random Forest and JRip both achieved 98.00% accuracy and 0.980 weighted F1. Their Macro-F1 scores were 0.947 and 0.945, respectively, and both achieved a HIGH-class recall of 0.935, substantially outperforming OneR and AdaBoost. For consistent comparison across the reported model summaries, Table IV reports  $1 - R_H$  as an upper bound on CUR according to Eq. (9).

TABLE IV  
OVERALL CLASSIFICATION AND SECURITY-SENSITIVE PERFORMANCE

| Model         | Acc. (%)     | W-F1         | M-F1         | High R       | CUR UB↓      |
|---------------|--------------|--------------|--------------|--------------|--------------|
| OneR          | 90.20        | 0.885        | 0.665        | 0.446        | $\leq 0.554$ |
| AdaBoost      | 93.20        | 0.921        | 0.754        | 0.748        | $\leq 0.252$ |
| Random Forest | <b>98.00</b> | <b>0.980</b> | <b>0.947</b> | <b>0.935</b> | $\leq 0.065$ |
| <b>JRip</b>   | <b>98.00</b> | <b>0.980</b> | 0.945        | <b>0.935</b> | $\leq 0.065$ |

W-F1: weighted F1-score; M-F1: Macro-F1; High R: HIGH-class recall; CUR UB: conservative upper bound on the HIGH-to-LOW critical underclassification rate, computed as  $1 - R_H$  for consistent cross-model comparison.

### B. Class-Wise Performance Analysis

Table V presents precision, recall, and F1-score for each risk class. Random Forest and JRip obtain the same MEDIUM-risk F1-score (0.908), while JRip gives a slightly higher LOW-class F1-score (0.989 versus 0.988) and Random Forest gives the higher HIGH-class F1-score (0.945 versus 0.939).

TABLE V  
CLASS-WISE PRECISION, RECALL, AND F1-SCORE

| Model         | LOW P/R/F1            | MEDIUM P/R/F1         | HIGH P/R/F1           |
|---------------|-----------------------|-----------------------|-----------------------|
| OneR          | .904/.992/.946        | .818/.310/.450        | .912/.446/.599        |
| AdaBoost      | .931/.995/.962        | .931/.310/.466        | .945/.748/.835        |
| Random Forest | .983/.994/.988        | <b>.974/.851/.908</b> | <b>.956/.935/.945</b> |
| <b>JRip</b>   | <b>.984/.994/.989</b> | <b>.974/.851/.908</b> | .942/.935/.939        |

P: precision; R: recall; F1: F1-score.

Figure 2 confirms that Random Forest and JRip maintain stronger and more balanced performance across the three risk classes.

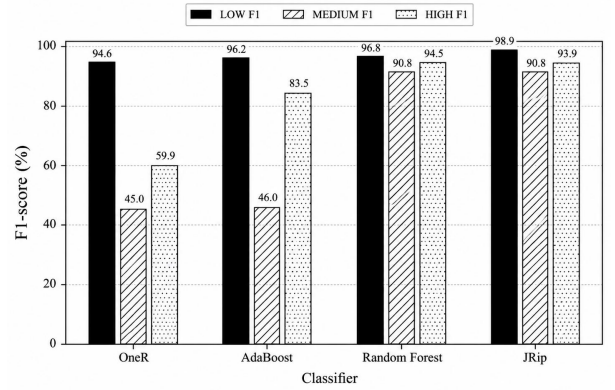


Fig. 2. Class-wise F1-score comparison across the evaluated classifiers.

a) *Security-Sensitive Error Analysis:* The conservative CUR upper bound,  $1 - R_H$ , decreases from 0.554 for OneR and 0.252 for AdaBoost to 0.065 for both Random Forest and JRip. These values bound all HIGH-class errors and provide a consistent comparison across the evaluated models.

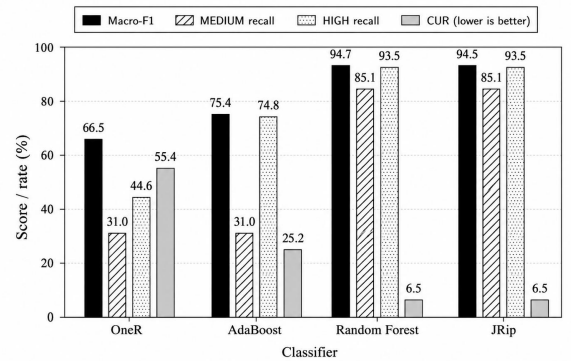


Fig. 3. Security-sensitive comparison. Higher Macro-F1 and class recall are preferred. The bars labeled CUR visualize the conservative upper bound  $1 - R_H$  for consistent cross-model comparison.

b) *Error Distribution*: Random Forest and JRip each identify 74/87 MEDIUM and 130/139 HIGH samples, compared with 27/87 MEDIUM and 62/139 HIGH for OneR, and 27/87 MEDIUM and 104/139 HIGH for AdaBoost. These results confirm their stronger minority-class performance.

Overall, Random Forest and JRip provide the strongest reported performance. Both achieve 98.00% accuracy and 0.980 weighted F1, while Random Forest has a slightly higher Macro-F1 (0.947 versus 0.945). Because the reported summaries do not establish a statistically significant difference, the proof-of-concept treats both as candidate runtime predictors rather than claiming a unique best model.

### C. Security and Decryption Verification

The security-oriented results verify classifier fidelity to the synthetic reference policy and quantify HIGH-class underclassification through the conservative CUR upper bound. At the policy layer, the decryption budget is fixed at seven fields for LOW risk, two fields for MEDIUM risk, and zero fields for HIGH risk, with  $N_{\text{dec}}(q) \leq B_{\text{dec}}(\hat{c}_q)$  as specified in Eq. (13). Thus, a HIGH-risk prediction is mapped to denial before protected-field decryption, whereas LOW- and MEDIUM-risk requests remain subject to their corresponding exposure limits.

The reported experiments do not directly measure plaintext-exposure reduction in a deployed backend. Implementation-level properties, including fail-closed enforcement, ciphertext-only ORM retrieval, AES-GCM decryption execution, and decryption-budget checks, require corresponding runtime security-test records before being reported as experimentally verified.

## V. CONCLUSION

This paper addressed unnecessary plaintext exposure in backend systems by proposing a risk-adaptive late-binding decryption framework that keeps protected fields encrypted after ORM retrieval and applies mandatory authorization checks before determining the permitted exposure level. In the controlled synthetic evaluation, Random Forest and JRip both achieved 98.00% accuracy and a weighted F1-score of 0.980, with Macro-F1 scores of 0.947 and 0.945, respectively. Both models obtained HIGH-class recall of 0.935, giving a conservative CUR upper bound of 0.065 for consistent cross-model comparison. These findings support the feasibility of contextual policy approximation, while the actual reduction of plaintext exposure and the runtime security invariants require end-to-end implementation testing. Future work will evaluate the framework using real operational access traces and complete runtime security verification.

## REFERENCES

- [1] Joint Task Force, "Security and privacy controls for information systems and organizations," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-53 Rev. 5, Sep. 2020.
- [2] National Institute of Standards and Technology, "Advanced encryption standard (AES)," National Institute of Standards and Technology, Federal Information Processing Standards Publication 197, May 2023.
- [3] M. U. Sardar and C. Fetzter, "Confidential computing and related technologies: A critical review," *Cybersecurity*, vol. 6, no. 1, p. 10, 2023.
- [4] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-38D, Nov. 2007.
- [5] Eclipse Foundation, "Jakarta persistence 3.2 specification," Jakarta EE Specification, 2024, release for Jakarta EE 11.
- [6] E. Costa, J. P. L. de Siqueira, C. E. Pantoja, and N. M. Lazarin, "Integrity: An object-relational framework for data security," in *Proceedings of the 24th International Conference on Enterprise Information Systems (ICEIS)*, vol. 1, 2022, pp. 259–266.
- [7] D. F. Ferraiolo and D. R. Kuhn, "Role-based access controls," in *Proceedings of the 15th National Computer Security Conference*, Baltimore, MD, USA, 1992, pp. 554–563.
- [8] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [9] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to attribute based access control (ABAC) definition and considerations," National Institute of Standards and Technology, NIST Special Publication 800-162, 2014.
- [10] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," National Institute of Standards and Technology, NIST Special Publication 800-207, 2020.
- [11] S. Kandala, R. S. Sandhu, and V. Bhamidipati, "An attribute based framework for risk-adaptive access control models," in *2011 Sixth International Conference on Availability, Reliability and Security*. IEEE, 2011, pp. 236–241.
- [12] R. A. Shaikh, K. Adi, and L. Logrippo, "Dynamic risk-based decision methods for access control systems," *Computers & Security*, vol. 31, no. 4, pp. 447–464, 2012.
- [13] H. Khambhammettu, S. Boulares, K. Adi, and L. Logrippo, "A framework for risk assessment in access control systems," *Computers & Security*, vol. 39, pp. 86–103, 2013.
- [14] H. F. Atlam, M. A. Azad, M. O. Alassafi, A. A. Alshdadi, and A. Alenezi, "Risk-based access control model: A systematic literature review," *Future Internet*, vol. 12, no. 6, p. 103, 2020.
- [15] E. Shmueli, R. Vaisenberg, Y. Elovici, and C. Glezer, "Database encryption: An overview of contemporary challenges and design considerations," *ACM SIGMOD Record*, vol. 38, no. 3, pp. 29–34, 2010.
- [16] R. A. Popa, C. M. S. Redfield, N. Zeldovich, and H. Balakrishnan, "CryptDB: Protecting confidentiality with encrypted query processing," in *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*. ACM, 2011, pp. 85–100.
- [17] D. R. dos Santos, R. Marinho, G. R. Schmitt, C. M. Westphall, and C. B. Westphall, "A framework and risk assessment approaches for risk-based access control in the cloud," *Journal of Network and Computer Applications*, vol. 74, pp. 86–97, 2016.
- [18] V. K. Matter, M. G. Martins, and J. L. V. Barbosa, "Context-aware security and machine learning for access control: A systematic mapping and taxonomies," *Computer Science Review*, vol. 60, p. 100880, 2026.
- [19] V. K. Matter, T. de Carvalho Schardosim, M. G. Martins, and J. L. V. Barbosa, "Anubis: A smart context-aware security model for access control," *Journal of Information Security and Applications*, vol. 100, p. 104469, 2026.
- [20] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I. H. Witten, "The WEKA data mining software: An update," *ACM SIGKDD Explorations Newsletter*, vol. 11, no. 1, pp. 10–18, 2009.
- [21] R. C. Holte, "Very simple classification rules perform well on most commonly used datasets," *Machine Learning*, vol. 11, no. 1, pp. 63–90, 1993.
- [22] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [23] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
- [24] W. W. Cohen, "Fast effective rule induction," in *Proceedings of the Twelfth International Conference on Machine Learning*. Morgan Kaufmann, 1995, pp. 115–123.